

set, only that items are added that are hashable, just like set, but lists are unhashable

Hashable means Python can calculate a fixed hash value for an object so it can be stored efficiently in a set or used as a dictionary key.

Non-mandatory exercises: I

strings, sets, are hashable, but list, dict is unhashable because it can be changed

Sets and dictionaries

1. Which of the following lines will result in an error? Why will they result in an error? How could you correct the code?

☐ `a = {}
a["hi"] = "hello"`

Syntax Meaning
{ } Empty dictionary
set() Empty set
() Empty tuple
{1, 2, 3} Set
(1, 2, 3) Tuple
{"a": 1} Dictionary

☐ `b = {}
b.add("hi")`

Dictionaries do not have .add(). Use `b["hi"] = "hello"`.

☐ `c = []
c.add("hi")`

use `append()`

☐ `d = set()
d.add("hi")`

works for sets

☐ `e = []
e[0] = "zero"`

it does not have any syntax, you can change like this, but not add new

☐ `f = {}
f[0] = "zero"`

Dictionary can assign a new key: `f[0] = "zero"`.

☐ `g = set()
g[0] = "zero"`

sets cannot be indexed with `[0]`

☐ `h = ["zero"]
h[0] = "ZERO"`

☐ `i = {0: "zero"}
i[0] = "ZERO"`

☐ `j = set()
j.add(("hi", "hello"))`

☐ `k = {("hi", "hello")}`

This creates a set containing one tuple.

☐ `l = set()
l.add(["hi", "hello"])`

A list is mutable/unhashable, so it cannot be a set element.

☐ `m = {["hi", "hello"]}`

dictionary needs a colon `:` to separate key and value, so it's a set.
A list cannot be stored inside a set.

Non-mandatory exercises: I

- ☐ `n = {"hi": "hello"}` A dictionary cannot be an element of a set.
- ☐ `o = {"hi"}` `0=`
- ☐ `p = {"hi", "hello"}` same `0=`
- ☐ `r = set()`
`r.add("one", "two")`
- ☐ `s = set()`
`s.add("one", "two")` .add() accepts one argument, but two are given.
- ☐ `t = {"hi" : "hello"}`
- ☐ `u = {"hi", "hello": "hej"}` A tuple can be used as a dictionary key.
- ☐ `x = [{"hi", "hello": "hej"}]` List cannot be a dictionary key because it is unhashable.
- ☐ `y = {"hej" : ["hi", "hello"]}` A list can be a dictionary value.
- ☐ `z = {"hej" : {"hi", "hello"}}` A set cannot be used as a dictionary key.
- ☐ `a1 = {"hej" : {"hi", "hello"}}`
- ☐ `a2 = {tuple(["hi", "hello"]): "hej"}` tuple can be key, but list and set cannot be
- ☐ `a3 = {}`
`a3[("hi", "hello")] = "hej"`
- ☐ `a4 = {}`
`a4[["hi", "hello"]] = "hej"` list cannot be key
- ☐ `a5 = {}`
`a5["hello"] = {"formal": "hej", "colloquial": "tja"}` A dictionary can be a dictionary value.
- ☐ `a6 = {"hej" : "hello"}`
`a6["hej"].append("hi")` `a6[hej]` will give hello, and hello is a string, and strings do not use append
- ☐ `a7 = {"hej" : ["hello"]}`
`a7["hej"].append("hi")`

Non-mandatory exercises: I

☐ `a8 = {"hej" : {"hello"}}
a8["hej"].add("hi")`

☐ `a9 = {"hej" : "hello"}
print(a9["hello"])`

"hello" is not a key. This causes `KeyError`.

☐ `a10 = {"hej" : "hello"}
print(a9["hej"])`

A set is mutable, meaning you can change it after creating it: so we can't use it as a dictionary key

Non-mandatory exercises: I

2. Which of the following prints True? (None of them results in an error. Some of them print False, and some don't print anything at all.)

(The backslash in

```
print({"a", "b", "c"}.intersection({"b", "c", "d"})\
      == {"c", "c", "b", "b"})
```

means that the code continues on the following line.)

true, Both are sets containing the same two elements.

- ☐ `print({"hej", "hello"} == set(("hej", "hello")))` true, Both are sets containing the same two elements.
- ☐ `print({"hej", "hello"} == {"hej": "hello"})` False because one is a set and the other is a dictionary.
- ☐ `print({} == set())` False.empty dictionary, while set() is an empty set.
- ☐ `print({"hej": "hello"} == {"hello": "hej"})` no, because the key-value pairs are different.
- ☐ `print(type({"hej": "hello"}) == type({"hello": "hej"}))` true
- ☐ `print("hej" in {"hej": "hi", "hejdå": "goodbye"})` true
- ☐ `print("hi" in {"hej": "hi", "hejdå": "goodbye"})` false, because it's a value
- ☐ `print("hi" in {"hej": "hi"}.values())` true
- ☐ `print("hi" in {"(hi", "hello)": "hej"})` False because "hi" is not the key. The key is: (hi, hello)
- ☐ `if {}:`
 `print(True)` Nothing prints because an empty dictionary is False in a Boolean context.
- ☐ `if set():`
 `print(True)` Nothing prints because an empty set is False in a Boolean context.
- ☐ `print(not {})` true
- ☐ `print(not set())` true
- ☐ `while {}:`
 `print(True)` Nothing prints because {} is false.
- ☐ `print(set({"a": 1, "b": 2}.keys()) == {"a", "b"})` true

Non-mandatory exercises: I

- `print(set({"a": 1, "b" : 2}.values()) == {1, 2})` `true`
- `print({"a": 1, "b" : 2}.keys == {"a", "b"})` `False` because `.keys` refers to the method itself. It needs `()`.
- `print({"a": 1, "b" : 2}.values == {1, 2})` `False`, is the method, not the values.
- `print({"a", "a", "b"} == {"a", "b", "b"})` `true`, Sets automatically remove duplicates.
- `print(len(("a", "a", "b")) == len({"a", "a", "b"}))` `false`, tuple = 3, set=2
- `print({"a", "b", "c"}.intersection({"b", "c", "d"}) \
== {"c", "c", "b", "b"})`    `false`, it will be b,c only
- `print({"a", "b", "c"}.intersection({"b", "c", "d"}).
intersection({"a", "b"}) \
== {"b"})`    `true`
- `print({"a", "b", "c"}.union({"b", "c", "d"}) \
== {"a", "b", "c", "d"})`    `true`
- `print(len(({1, 2}, {3, 4}).union({5, 6}))) == 6)` `it will be 3, so false`
- `print(not ({"a", "b"}.intersection({"c", "d"})))` `true`
- `print(not ({"a", "b"}.union({"c", "d"})))` `false`

Non-mandatory exercises: I

Using sets and dictionaries

3. The code below is supposed to return tokenised sentences that are censored.

```
NOT_ALLOWED = {"jävligt", "fan"}

def censure(tokenised_sent):
    censored_sent = set()           we can make it list, so order can be preserved
    for word in tokenised_sent:
        if word not in NOT_ALLOWED:
            censored_sent.add(word)  list will use .append
    return censored_sent

for s in [
    ["det", "här", "var", "jävligt", "gott"],
    ["fan", "det", "bästa", "jag", "ätit"]]:
    print(censure(s))
```

Should print:

```
['det', 'här', 'var', 'gott']
['det', 'bästa', 'jag', 'ätit']
```

Instead, the following is printed:

```
{'gott', 'var', 'det', 'här'}
{'jag', 'ätit', 'det', 'bästa'}
```

A. How could you fix the code, to make it work?

B. Let's say that NOT_ALLOWED would be contains a really large amount of words. Would the best data type to save it then be a list or a set?

C. Lets say that you don't want to just remove the words that are censored, but replace them with something else. Let's say you want to replace "jävligt" with "väldigt", and "fan" with "attans". What combined data type would then be a good for storing this information. (And how would you implement the code?)

B. Use a set: Especially when there are thousands of words.

We repeatedly ask: if word not in NOT_ALLOWED:

Sets are designed for efficient membership checking.

c. A dictionary is suitable:

```
REPLACEMENTS = {
    "jävligt": "väldigt",
    "fan": "attans"
}
```

```
c. def censure(tokenised_sent):
    result = []
    for word in tokenised_sent:
        if word in REPLACEMENTS:
            result.append(REPLACEMENTS[word])
        else:
            result.append(word)
    return result
```

Non-mandatory exercises: I

4. The following code works fine. What will it do?

```
def test_user(dictionary, reversed):
    nr_of_correct = 0
    for key, value in dictionary.items():
        if not reversed:
            expected_word = value
            ask_for = key
        else:
            print("Not implemented")
            return None

    answer = input(ask_for + "? ")
    if answer == expected_word:
        nr_of_correct += 1
    return nr_of_correct
```

replace:

else:

expected_word = key
ask_for = value

```
en_sw = {"hello": "hej",
         "goodbye": "hejdå", "sorry": "förlåt"}
nr_of_correct_answered = test_user(en_sw, False)
```

However, if you add the following code, which sets the reversed flag to `True`, the code will just print "Not implemented".

```
print(nr_of_correct_answered)
nr_of_correct_answered = test_user(en_sw, True)
print(nr_of_correct_answered)
```

- A. What code do you need to add, to be able to do a glossary test from Swedish to English?
- B. The code does not notify the user that an incorrect answer has been provided. What code would you add to be able to provide this user feedback?
- C. Do you need to do any changes in your code, to be able to test the user on other word pairs in Swedish and English, or with words from another language pair?

B: if answer == expected_word:

nr_of_correct += 1

print("Correct!")

else:

print("Incorrect! The correct answer is:", expected_word)

c. No major changes are needed.

The function uses the dictionary passed to it, so you can use another dictionary containing other language pairs.



Non-mandatory exercises: I

5. A list of tuples stores **word pairs in English and Swedish**. You want to store it in two dictionaries instead. So that the code below prints:

```
{'coffee': 'kaffe', 'tea': 'te', 'juice': 'jos', 'hot chocolate': 'varm choklad'}  
{'kaffe': 'coffee', 'te': 'tea', 'jos': 'juice', 'varm choklad': 'hot chocolate'}
```

However, two lines of code is missing.

- A. What **two code lines** are needed?

```
words = [("coffee", "kaffe"), ("tea", "te"), ("juice", "jos"),  
         ("hot_chocolate", "varm_choklad")]
```

```
english_to_swedish = {}  
swedish_to_english = {}
```

```
for english, swedish in words:  
    # Two lines of code are missing here  
    |   english_to_swedish[english]=swedish  
    |   english_to_swedish[swedish]=english  
print(english_to_swedish)  
print(swedish_to_english)
```

- B. What code the reason be to store the content in a dictionary instead in a list of tuples?
(Imagine you had a lot more words in you list of tuples than those in this example.)

A dictionary gives direct lookup using a key. This becomes especially useful when you have a large number of words.

Non-mandatory exercises: I

6. A. What does the following code do? (The backslash in

```
words = \
    text.replace(',', ' ').replace('.', ' ').lower().split()
```

means that the code continues on the following line.)

```
meta = ("One morning, when Gregor Samsa woke from troubled
dreams, he found himself transformed in his bed into a
horrible vermin. He lay on his armour-like back, and if he
lifted his head a little he could see his brown belly,
slightly domed and divided by arches into stiff sections")
```

```
def count_words(text):
```

```
    words = \
        text.replace(',', ' ').replace('.', ' ').lower().split()
    counter = {}
```

```
    for w in words:
        if w not in counter:
            counter[w] = 1
        else:
            counter[w] += 1
            #means: counter[w] = counter[w] + 1
    return counter
```

A. The program counts how many times each word appears in a text.

```
m_freqs = count_words(meta)
```

```
while True: # The user stops with Ctrl-C
    user_word = input("Write a word: ")
    print("The frequency is: " + str(m_freqs.get(user_word, 0)))
    #print("The frequency is: " + str(m_freqs[user_word]))
```

B. The programmer was not happy with this code line:

```
print("The frequency is: " + str(m_freqs[user_word]))
```

What was the problem with this code line, why was the code line above used instead?

C. It would also have been possible to keep this code line, but to put inside an if-else-clause to achieve the functionality as using `get()`. What would this if-else-clause look like?

b. `m_freqs[user_word]`

If the user enters a word that doesn't exist in the dictionary, Python gives: `KeyError` so, the 2nd option (Give me the frequency if the word exists. Otherwise give me 0.)

```
c. if user_word in m_freqs:
    print("The frequency is: " + str(m_freqs[user_word]))
else:
    print("The frequency is: 0")
```

A. tuple, as key, and string as values, because coded is returned by: `code_word(word)` and is a tuple containing two strings.

B. to reverse: `fikon_word[word]=coded`

C.

Non-mandatory exercises: I

7. The function `code_word()`, takes a word as input parameter, and returns a tuple with two strings, which is the “translation” of the input word into fikonspråket. For instance the argument `"stump"`, will get result in the tuple `("fimp", "stukon")`, being returned.

A. What data type is used as keys in `fikon_words`, and what data type is used as values?

B. Now the fikonspråket translation of a word is used as key, and the original word is used as value. How would you change the code, if you would do it the other way around. I.e., the original word as key, and the fikonspråket-coded version as value?

C. What if you just wanted to store all unique string that are created when the transformation to fikonspråket is carried out. How would you change the code to achieve this? I.e., the following is to be collected: `'slukon', 'fika', 'fimp', 'filociped', 'kakon', 'bakon', 'stukon', 'vekon'` (You find the answer on the following page.)

```
def get_index_first_vowel(word):
    vowels = {"a" , "e" , "i" , "o" , "u" , "y", "å", "ä", "ö" }
    for i in range(0, len(word)):
        if word[i] in vowels:
            return i

def code_word(word):
    index_first_vowel = get_index_first_vowel(word)  takes index of first vowel
    if not index_first_vowel:  if there is no index
        return word, ""
    split_at = index_first_vowel + 1 # Split after first vowel
    first_word = "fi" + word[split_at:]
    second_word = word[:split_at] + "kon"

    return first_word, second_word

fikon_words = {}
for word in ["stump", "kaka", "velociped", "baka", "slump"]:
    coded = code_word(word)
    fikon_words[coded] = word

print(fikon_words)
```

c. `fikon_set = set()`

```
for word in ["stump", "kaka", "velociped", "baka", "slump"]:
    first, second = code_word(word)
```

```
    fikon_set.add(first)
    fikon_set.add(second)
```

```
print(fikon_set)
```

because a set automatically removes the duplicate

Non-mandatory exercises: I

Answer, C:

```
fikon_set = set()
for word in ["stump", "kaka", "velociped", "baka", "slump"]:
    first, second = code_word(word)
    fikon_set.add(first)
    fikon_set.add(second)
print(fikon_set)
```

D. The first two lines of code below prints the following:

```
{('fig', 'jakon'): 'jag',
 ('fikar', 'bakon'): 'bakar',
 ('fin', 'ekon'): 'en',
 ('fika', 'bakon'): 'baka',
 ('fit', 'dekon'): 'det',
 ('fir', 'äkon'): 'är',
 ('fingen', 'ikon'): 'ingen',
 ('fimp', 'slukon'): 'slump'}
```

and

```
['fig', 'jakon', 'fikar', 'bakon', 'fin', 'ekon', 'fika',
 'bakon', 'fit', 'dekon', 'fir',
 'äkon', 'fingen', 'ikon', 'fimp', 'slukon']
```

Can you describe what the rest of the code does?

```
print(fikon_words)
print(coded_sentence)
for i in range(0, len(coded_sentence), 2):
    print(fikon_words[(coded_sentence[i], coded_sentence[i+1])])
```

D. So the loop takes every two coded strings, combines them into a tuple, uses that tuple as a dictionary key, and retrieves the original word.

Non-mandatory exercises: I

You have written this code, that will let the user input content to a dictionary with English words as keys and Swedish words as values.

```
english_swedish_dictionary = {}
print("Write 'Q' to break")
while True:
    english = input("Type an English word: ")
    if english == "Q":
        break

    swedish = input("Type the Swedish translation: ")
    if swedish == "Q":
        break

    english_swedish_dictionary[english] = swedish

print(english_swedish_dictionary)
```

However, the user is not happy. First they wrote the word "coffee", with a translation. Then they wrote the English word "both", and then the translation "båda". Thereafter they wrote the English word "both" again, but this time with "bägge" as the translation. The user expected both to be saved, but the dictionary looks like this:

```
{'coffee': 'kaffe', 'both': 'bägge'}
```

How would you rewrite the code, so that it instead saves a list of different possible translations into Swedish. I.e., a list that looks like this:

```
{'coffee': ['kaffe'], 'both': ['båda', 'bägge']}
```

(Solution on the following page.)

```
if english in dict.keys():
```

Non-mandatory exercises: I

Answers

```
english_swedish_dictionary = {}
print("Write 'Q' to break")
while True:
    english = input("Type an English word: ")
    if english == "Q":
        break

    swedish = input("Type the Swedish translation: ")
    if swedish == "Q":
        break

    if english not in english_swedish_dictionary:
        english_swedish_dictionary[english] = []

    english_swedish_dictionary[english].append(swedish)

print(english_swedish_dictionary)
```